



Senior Engineer Assessment: Banking Transaction Categorization

Overview

This assessment is designed to evaluate your skills as a Senior Engineer. You are tasked with creating a system that categorises banking transactions into predefined categories.

Requirements

- Implement a system for accepting and categorising banking transaction history (to be provided in the form of a CSV file)
 - Use 10 predefined transaction categories
 - Create an API for submitting transactions
 - Create an API for retrieving categorised transactions
- Use AI-powered categorisation with any AI coding tool of your choice, include the prompt(s) you used in your repository
- Choose the programming language and framework you are most comfortable with and briefly explain your choice in the documentation
- Package the solution in Docker, runnable locally via docker-compose
- Include documentation with trade-offs in the repository
- Implement tests for the solution

Predefined Categories

Use the following 10 categories for transaction classification:

1. Groceries
2. Dining Out
3. Utilities
4. Transportation
5. Entertainment
6. Healthcare
7. Shopping

8. Housing
9. Education
10. Miscellaneous

Technical Stack

- Backend: the language and framework of your choice (document your reasoning)
- Database: your choice (e.g. MongoDB, PostgreSQL, etc.)
- Containerisation: Docker

API Endpoints

Implement the following API endpoints:

- **POST /transactions:** Submit a new transaction
- **GET /transactions:** Retrieve all categorised transactions
- **GET /transactions/:id:** Retrieve a specific transaction by ID

Docker Setup

Ensure your solution can be run locally using Docker Compose. Include a *docker-compose.yml* file in your repository.

Documentation

Provide clear documentation in your repository, including:

- Setup and running instructions
- API documentation
- Explanation of the categorisation algorithm and the AI prompt(s) used
- Trade-offs and design decisions
- Your reasoning for the chosen language and framework

Testing

Implement unit tests and integration tests for your solution. Aim for good test coverage of critical components.

Evaluation Criteria

Your submission will be evaluated based on:

- Functionality and completeness of the solution
- Code quality and organisation
 - Small and logical commits to follow the steps taken
- Effective use of the chosen programming language
- Docker configuration and ease of setup
- Quality of documentation and explanation of trade-offs
- Test coverage and quality
- Efficient and thoughtful use of AI coding tools

Submission

Please submit your solution as a GitHub repository link. Ensure that your repository is public or provide access to the reviewers.

Time Limit

You should aim to complete this assessment within the time of the interview, but that's fine if not. Quality is more important than quantity, so focus on delivering a well-structured solution rather than implementing every possible feature.

Good luck with your assessment! We look forward to reviewing your solution.